ASSIGNMENT NO: 2

Network Lab Experiments

NAME:AMALJITH M

Class: S5, CSE A

Roll No: 10

Date: 08/08/2026

Question 1: Hello – Client/Server Communication

Aim

To implement a basic client–server application using TCP sockets in C, in which the client connects to the server, sends a greeting message, and receives a reply from the server.

Client Code

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>

#define PORT 8080

#define BUFFER_SIZE 1024

int main() {

   int server_fd, client_socket;
```

```
struct sockaddr_in address;
int opt = 1;
int addrlen = sizeof(address);
char buffer[BUFFER_SIZE] = {0};
char *reply = "Hello Client";

// 1. socket()
server_fd = socket(AF_INET, SOCK_STREAM, 0);
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);

// 2. bind()
bind(server_fd, (struct sockaddr *)&address, sizeof(address));

// 3. listen()
listen(server_fd, 3);
printf("Server listening on port %d...\n", PORT);

// 4. accept()
client_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t*)&addrlen);

// 5. recv() - Read message from client
recv(client_socket, buffer, BUFFER_SIZE, 0);
```

```
    printf("Client says: %s\n", buffer);

    // 6. send() - Reply to client
    send(client_socket, reply, strlen(reply), 0);
    printf("Replied to client: %s\n", reply);

    close(client_socket);
    close(server_fd);
    return 0;
}
```

Server side code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE] = {0};
    char *message = "Hello Server";
```

```
    // 1. socket()
    sock = socket(AF_INET, SOCK_STREAM, 0);

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);

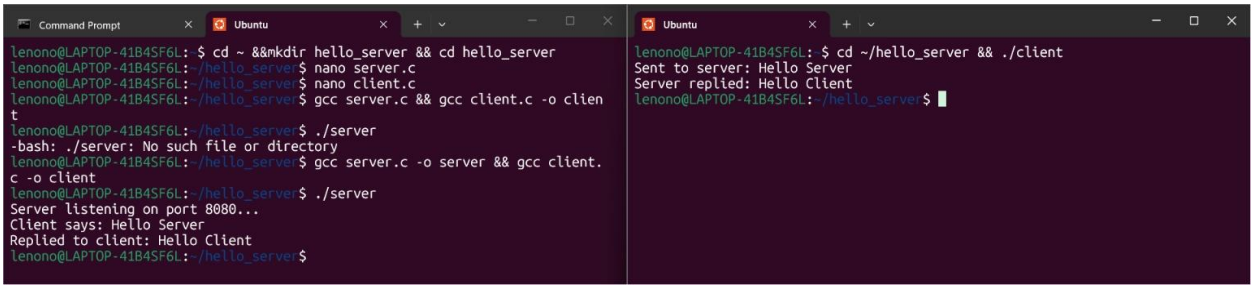
    // 2. connect()
    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        printf("Connection Failed!\n");
        return -1;
    }

    // 3. send() - Send message to server
    send(sock, message, strlen(message), 0);
    printf("Sent to server: %s\n", message);

    // 4. recv() - Receive reply from server
    recv(sock, buffer, BUFFER_SIZE, 0);
    printf("Server replied: %s\n", buffer);

    close(sock);
    return 0;
}
```

Output



Question 2: Arithmetic Server

Aim

To implement a client–server application in which the client sends two integers to the server, and the server computes and returns their sum, difference, product, and quotient.

```
#ifndef PROTOCOL_H

#define PROTOCOL_H


typedef struct {

    int num1;

    int num2;

} Request;


typedef struct {

    int add;

    int sub;

    int mul;

    float div;

    int is_div_zero;

} Response;
```

```
#define PORT 8080


#endif
```

Server code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include "protocol.h"

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in addr = { .sin_family = AF_INET, .sin_addr.s_addr = INADDR_ANY, .sin_port =
htons(PORT) };
    bind(server_fd, (struct sockaddr *)&addr, sizeof(addr));
    listen(server_fd, 3);

    printf("Server running on port %d...\n", PORT);

    while (1) {
        int sock = accept(server_fd, NULL, NULL);
        Request req;
```

```
        Response res;

        if (read(sock, &req, sizeof(req)) > 0) {
            res.add = req.num1 + req.num2;
            res.sub = req.num1 - req.num2;
            res.mul = req.num1 * req.num2;
            if (req.num2 != 0) {
                res.div = (float)req.num1 / req.num2;
                res.is_div_zero = 0;
            } else {
                res.div = 0.0f;
                res.is_div_zero = 1;
            }
            send(sock, &res, sizeof(res), 0);
        }
        close(sock);
    }
    return 0;
}
```

Client code

```
#include <stdio.h>
#include <unistd.h>
#include <arpa/inet.h>
#include "protocol.h"

int main() {
```

```
Request req;
Response res;

printf("Enter two integers: ");
scanf("%d %d", &req.num1, &req.num2);

int sock = socket(AF_INET, SOCK_STREAM, 0);
struct sockaddr_in serv_addr = { .sin_family = AF_INET, .sin_port = htons(PORT) };
inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);

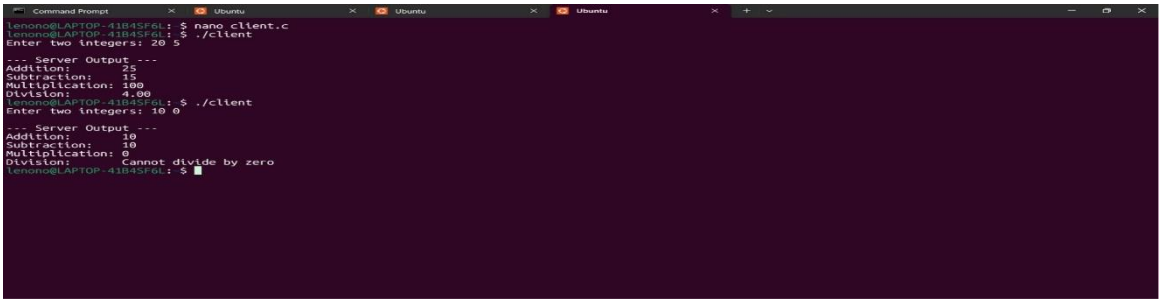
if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
    printf("Connection Failed!\n");
    return 1;
}

send(sock, &req, sizeof(req), 0);
read(sock, &res, sizeof(res));

printf("\n--- Server Output ---\n");
printf("Addition:       %d\n", res.add);
printf("Subtraction:    %d\n", res.sub);
printf("Multiplication: %d\n", res.mul);
if (res.is_div_zero) {
    printf("Division:       Cannot divide by zero\n");
} else {
    printf("Division:       %.2f\n", res.div);
```

```
    }
    close(sock);
return 0;
}
```





Question 3: String Processing Server

Aim

To implement a client–server application in which the client sends a string to the server, and the server computes the length of the string, converts it to uppercase, and reverses it, then returns the results to the client.

```
#ifndef PROTOCOL_H
#define PROTOCOL_H

#define MAX_SIZE 256
#define PORT 8080
```

```
typedef struct {
    char str[MAX_SIZE];
} StringRequest;

typedef struct {
    int length;
    char uppercase[MAX_SIZE];
    char reversed[MAX_SIZE];
} StringResponse;

#endif
```

Server code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <arpa/inet.h>
#include "protocol.h"

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
```

```
    struct sockaddr_in addr = { .sin_family = AF_INET, .sin_addr.s_addr = INADDR_ANY, .sin_port =
htons(PORT) };
    bind(server_fd, (struct sockaddr *)&addr, sizeof(addr));
    listen(server_fd, 3);

    printf("String Server running on port %d...\n", PORT);

    while (1) {
        int sock = accept(server_fd, NULL, NULL);
        StringRequest req;
        StringResponse res;

        if (read(sock, &req, sizeof(req)) > 0) {
            int len = strlen(req.str);
            res.length = len;

            // Convert to Uppercase
            for (int i = 0; i < len; i++) {
                res.uppercase[i] = toupper((unsigned char)req.str[i]);
            }
            res.uppercase[len] = '\0';

            // Reverse String
            for (int i = 0; i < len; i++) {
                res.reversed[i] = req.str[len - 1 - i];
            }
```

```
            res.reversed[len] = '\0';

            send(sock, &res, sizeof(res), 0);
        }
        close(sock);
    }
    return 0;
}
```

Client code

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include "protocol.h"

int main() {
    StringRequest req;
    StringResponse res;

    printf("Enter a string: ");
    fgets(req.str, sizeof(req.str), stdin);
    req.str[strcspn(req.str, "\n")] = 0;

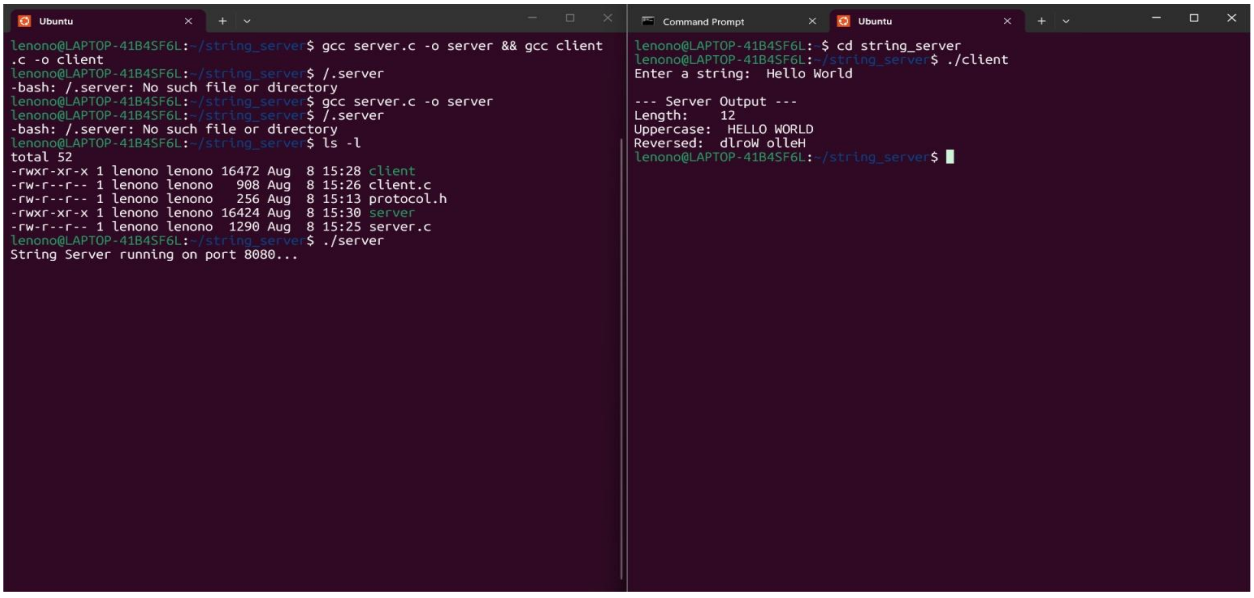
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in serv_addr = { .sin_family = AF_INET, .sin_port = htons(PORT) };
    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);
```

```
    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        printf("Connection Failed!\n");
        return 1;
    }

    send(sock, &req, sizeof(req), 0);
    read(sock, &res, sizeof(res));

    printf("\n--- Server Output ---\n");
    printf("Length:    %d\n", res.length);
    printf("Uppercase: %s\n", res.uppercase);
    printf("Reversed:  %s\n", res.reversed);

    close(sock);
    return 0;
}
```



Question 4: Array Processing Server

Aim

To implement a client–server application in which the client sends an array of N integers to the server, and the server computes the sum, average, maximum, and minimum of the array elements.

```
#ifndef PROTOCOL_H

#define PROTOCOL_H


#define MAX_SIZE 100

#define PORT 8080


typedef struct {

   int count;

   int arr[MAX_SIZE];

} ArrayRequest;


typedef struct {

   int sum;

   double average;

   int max;

   int min;

} ArrayResponse;


#endif
```

SERVER CODE

```
#include <stdio.h>
```

```
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include "protocol.h"

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in addr = { .sin_family = AF_INET, .sin_addr.s_addr = INADDR_ANY, .sin_port =
htons(PORT) };
    bind(server_fd, (struct sockaddr *)&addr, sizeof(addr));
    listen(server_fd, 3);

    printf("Array Processing Server running on port %d...\n", PORT);

    while (1) {
        int sock = accept(server_fd, NULL, NULL);
        ArrayRequest req;
        ArrayResponse res;

        if (read(sock, &req, sizeof(req)) > 0 && req.count > 0) {
            res.sum = 0;
            res.max = req.arr[0];
            res.min = req.arr[0];
```

```
        for (int i = 0; i < req.count; i++) {
            res.sum += req.arr[i];
            if (req.arr[i] > res.max) res.max = req.arr[i];
            if (req.arr[i] < res.min) res.min = req.arr[i];
        }

        res.average = (double)res.sum / req.count;

        send(sock, &res, sizeof(res), 0);
    }
    close(sock);
  }
  return 0;
}
```

Client code

```
#include <stdio.h>
#include <unistd.h>
#include <arpa/inet.h>
#include "protocol.h"

int main() {
  ArrayRequest req;
  ArrayResponse res;

  printf("Enter number of elements (N): ");
```

```
if (scanf("%d", &req.count) != 1 || req.count <= 0 || req.count > MAX_SIZE) {
    printf("Invalid size! Enter a number between 1 and %d.\n", MAX_SIZE);
    return 1;
}

printf("Enter %d integers space-separated: ", req.count);
for (int i = 0; i < req.count; i++) {
    scanf("%d", &req.arr[i]);
}

int sock = socket(AF_INET, SOCK_STREAM, 0);
struct sockaddr_in serv_addr = { .sin_family = AF_INET, .sin_port = htons(PORT) };
inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);

if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
    printf("Connection Failed!\n");
    return 1;
}

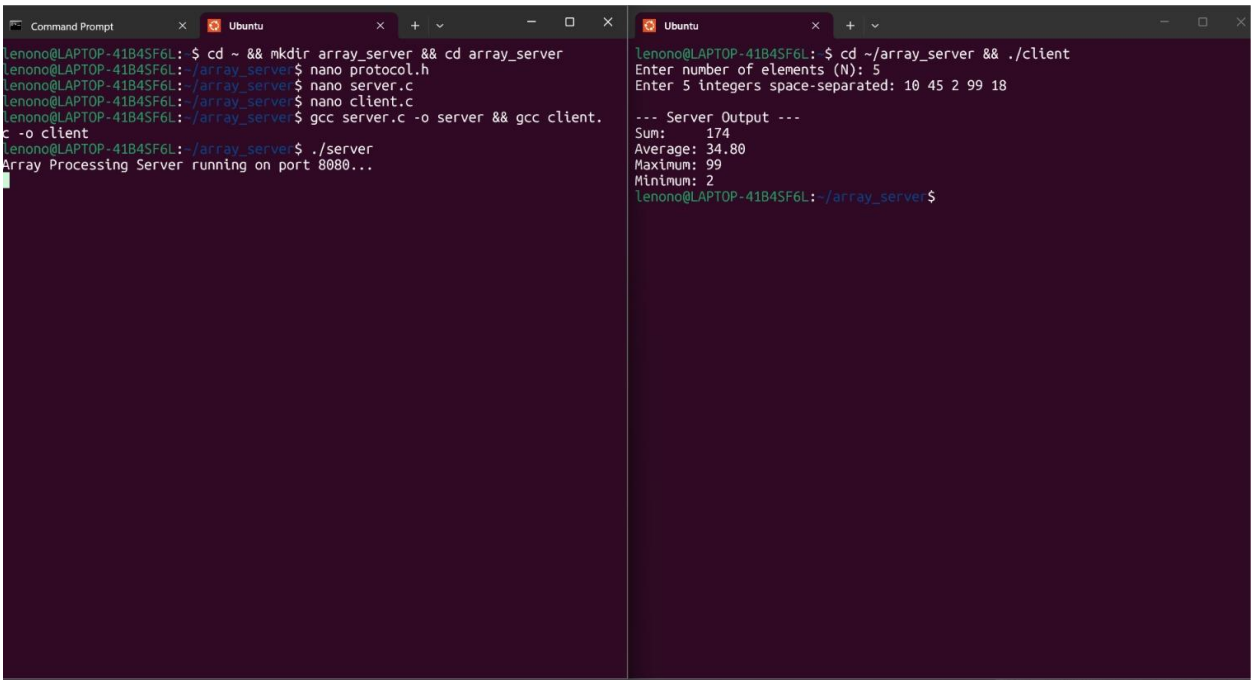
send(sock, &req, sizeof(req), 0);
read(sock, &res, sizeof(res));

printf("\n--- Server Output ---\n");
printf("Sum:     %d\n", res.sum);
printf("Average: %.2f\n", res.average);
printf("Maximum: %d\n", res.max);
```

```
    printf("Minimum: %d\n", res.min);


    close(sock);

    return 0;
}
```



Question 5: Matrix Analyzer

Aim

To implement a client–server application in which the client sends a square matrix to the server, and the server determines the type of matrix (diagonal, upper triangular, lower triangular, or general), and computes the sum of its elements and its trace.

```
#ifndef PROTOCOL_H

#define PROTOCOL_H


#define MAX_N 10

#define PORT 8080
```

```
typedef struct {
    int n;
    int matrix[MAX_N][MAX_N];
} MatrixRequest;

typedef struct {
    int sum;
    int trace;
    char type[50];
} MatrixResponse;

#endif
```

Server code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include "protocol.h"

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
```

```
    struct sockaddr_in addr = { .sin_family = AF_INET, .sin_addr.s_addr = INADDR_ANY, .sin_port =
htons(PORT) };
    bind(server_fd, (struct sockaddr *)&addr, sizeof(addr));
    listen(server_fd, 3);

    printf("Matrix Analyzer Server running on port %d...\n", PORT);

    while (1) {
        int sock = accept(server_fd, NULL, NULL);
        MatrixRequest req;
        MatrixResponse res;

        if (read(sock, &req, sizeof(req)) > 0 && req.n > 0) {
            res.sum = 0;
            res.trace = 0;
            int is_upper = 1;
            int is_lower = 1;

            for (int i = 0; i < req.n; i++) {
                for (int j = 0; j < req.n; j++) {
                    int val = req.matrix[i][j];
                    res.sum += val;

                    if (i == j) {
                        res.trace += val;
                    }
```

```
            if (i > j && val != 0) {
                is_upper = 0;
            }
            if (i < j && val != 0) {
                is_lower = 0;
            }
        }
    }

    if (is_upper && is_lower) {
        strcpy(res.type, "Diagonal Matrix");
    } else if (is_upper) {
        strcpy(res.type, "Upper Triangular Matrix");
    } else if (is_lower) {
        strcpy(res.type, "Lower Triangular Matrix");
    } else {
        strcpy(res.type, "General Matrix");
    }

    send(sock, &res, sizeof(res), 0);
  }
  close(sock);
 }
 return 0;
}
```

Client code

```
#include <stdio.h>
#include <unistd.h>
#include <arpa/inet.h>
#include "protocol.h"

int main() {
    MatrixRequest req;
    MatrixResponse res;

    printf("Enter matrix order N (max %d): ", MAX_N);
    if (scanf("%d", &req.n) != 1 || req.n <= 0 || req.n > MAX_N) {
        printf("Invalid matrix size!\n");
        return 1;
    }

    printf("Enter elements of %dx%d matrix row by row:\n", req.n, req.n);
    for (int i = 0; i < req.n; i++) {
        for (int j = 0; j < req.n; j++) {
            scanf("%d", &req.matrix[i][j]);
        }
    }

    int sock = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in serv_addr = { .sin_family = AF_INET, .sin_port = htons(PORT) };
    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);
```

```
    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        printf("Connection Failed!\n");
        return 1;
    }

    send(sock, &req, sizeof(req), 0);
    read(sock, &res, sizeof(res));

    printf("\n--- Server Analysis Output ---\n");
    printf("Matrix Type: %s\n", res.type);
    printf("Sum:         %d\n", res.sum);
    printf("Trace:       %d\n", res.trace);

    close(sock);
    return 0;
}
```

OUTPUT

